

1) You are given an array of integers `arr[]`. You have to reverse the given array. Note: Modify the array in place. Examples: Input: `arr = [1, 4, 3, 2, 6, 5]` Output: `[5, 6, 2, 3, 4, 1]` Explanation: The elements of the array are `[1, 4, 3, 2, 6, 5]`. After reversing the array, the first element goes to the last position, the second element goes to the second last position and so on. Hence, the answer is `[5, 6, 2, 3, 4, 1]`.

```

1 class Solution {
2     public void reverseArray(int arr[]) {
3         int left = 0;
4         int right = arr.length - 1;
5
6         while (left < right) {
7             // Swap elements
8             int temp = arr[left];
9             arr[left] = arr[right];
10            arr[right] = temp;
11
12            left++;
13            right--;
14        }
15    }
16 }

```

2) Given an integer array `arr[]` and an integer `k`, your task is to find and return the `k`th smallest element in the given array. Note: The `k`th smallest element is determined based on the sorted order of the array. Examples : Input: `arr[] = [10, 5, 4, 3, 48, 6, 2, 33, 53, 10]`, `k = 4` Output: 5 Explanation: 4th smallest element in the given array is 5.

```

1 import java.util.Arrays;
2 class Solution {
3     public int kthSmallest(int[] arr, int k) {
4         // Code here
5         Arrays.sort(arr);
6         return arr[k - 1];
7     }
8 }
9

```

3) You are given two arrays `a[]` and `b[]`, return the Union of both the arrays in any order. The Union of two arrays is a collection of all distinct elements present in either of the arrays. If an element appears more than once in one or both arrays, it should be included only once in the result. Note: Elements of `a[]` and `b[]` are not necessarily distinct. Note that, You can return the Union in any order

but the driver code will print the result in sorted order only. Examples: Input: $a[] = [1, 2, 3, 2, 1]$, $b[] = [3, 2, 2, 3, 3, 2]$ Output: $[1, 2, 3]$ Explanation: Union set of both the arrays will be 1, 2 and 3.

The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. It shows 'Test Cases Passed: 1111 / 1111', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 2 / 2', and 'Time Taken: 0.75'. Below this, it suggests 'Solve Next' problems: 'Intersection of Arrays with Distinct', 'LCM of given array elements', and 'Perfect Squares in a Range'. On the right, the code editor shows a Java solution for finding the union of two arrays. The code uses a HashSet to store elements from both arrays and then returns the union as a sorted ArrayList.

```

1 import java.util.*;
2
3 class Solution {
4     public static ArrayList<Integer> findUnion(int[] a, int[] b) {
5         HashSet<Integer> set = new HashSet<>();
6
7         // Add all elements from array a
8         for (int num : a) {
9             set.add(num);
10        }
11
12        // Add all elements from array b
13        for (int num : b) {
14            set.add(num);
15        }
16
17        // Return the union as an ArrayList
18        return new ArrayList<>(set);
19    }
20 }

```

4) You are given an integer array $arr[]$. You need to find the maximum sum of a subarray (containing at least one element) in the array $arr[]$. Note : A subarray is a continuous part of an array. Examples: Input: $arr[] = [2, 3, -8, 7, -1, 2, 3]$ Output: 11 Explanation: The subarray $[7, -1, 2, 3]$ has the largest sum 11.

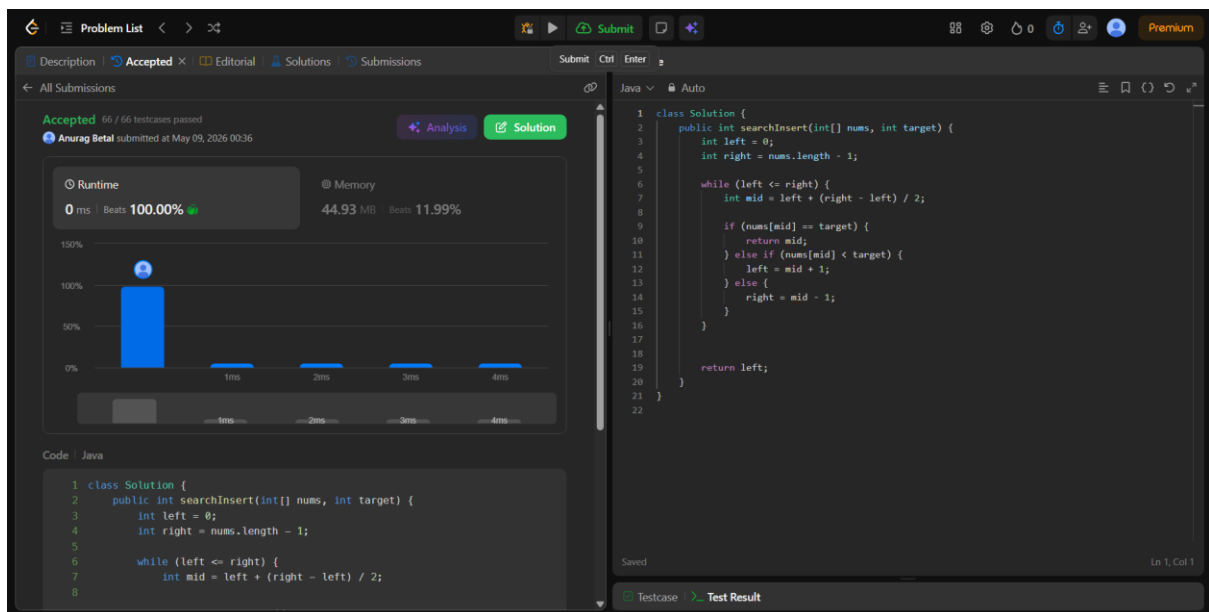
The screenshot shows a coding platform interface. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. It shows 'Test Cases Passed: 1120 / 1120', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 4 / 4', and 'Time Taken: 0.32'. Below this, it suggests 'Solve Next' problems: 'Count of Subarrays', 'Longest Arithmetic Subsequence', and 'Smallest sum contiguous subarray'. On the right, the code editor shows a Java solution for finding the maximum sum of a subarray. The code uses Kadane's algorithm to iterate through the array and keep track of the maximum sum ending at each position.

```

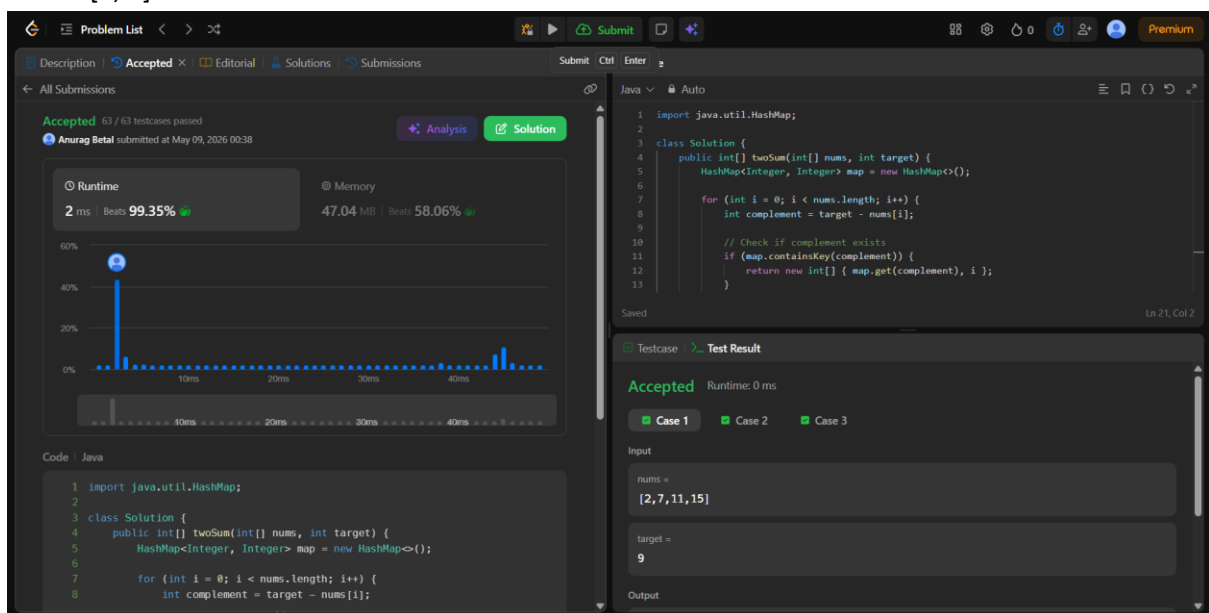
1 class Solution {
2     int maxSubarraySum(int[] arr) {
3         int maxEnding = arr[0];
4         int res = arr[0];
5
6         for (int i = 1; i < arr.length; i++) {
7             // Either extend the previous subarray or start new
8             maxEnding = Math.max(maxEnding + arr[i], arr[i]);
9
10            // Update result
11            res = Math.max(res, maxEnding);
12        }
13
14        return res;
15    }
16 }

```

5) Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order. You must write an algorithm with $O(\log n)$ runtime complexity. Example 1: Input: $nums = [1, 3, 5, 6]$, $target = 5$ Output: 2



6) Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order. Example 1: Input: `nums = [2,7,11,15]`, `target = 9` Output: `[0,1]` Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.



7) You are given an array `arr[]` of non-negative numbers. Each number tells you the maximum number of steps you can jump forward from that position. For example: • If `arr[i] = 3`, you can jump to index `i + 1`, `i + 2`, or `i + 3` from position `i`. • If `arr[i] = 0`, you cannot jump forward from that position. Your task is to find the minimum number of jumps needed to move from the first position in the array to the last position. Note: Return -1 if you can't reach the end of the array. Examples : Input: `arr[] = [1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9]` Output: 3 Explanation: First jump from 1st element to 2nd element with value 3. From here we jump to 5th element with value 9, and from here we will jump to the last.

Problem Solved Successfully ✓

Test Cases Passed: **1120 / 1120**

Attempts: Correct / Total: **2 / 2**

Accuracy: 100%

Time Taken: **0.26**

You get marks only for the first correct submission if you solve the problem without viewing the full solution.

Solve Next

Maximum Index, Jump Game, Wine Buying and Selling

Stay Ahead With:

```

1 class Solution {
2     public int minJumps(int[] arr) {
3         // code here
4         int n = arr.length;
5         if (n == 1) {
6             return 0;
7         }
8         if (arr[0] == 0) {
9             return -1;
10        }
11        int maxReach = arr[0];
12        int steps = arr[0];
13        int jumps = 1;
14        for (int i = 1; i < n; i++) {
15            if (i == n - 1) {
16                return jumps;
17            }
18            maxReach = Math.max(maxReach, i + arr[i]);
19            steps--;
20            if (steps == 0) {
21                jumps++;
22                if (i >= maxReach) {
23                    return -1;
24                }
25                steps = maxReach - i;
26            }
27        }
28        return -1;
29    }
30 }

```

8) Given an array `arr[]`. Your task is to find the minimum and maximum elements in the array.
 Examples: Input: `arr[] = [1, 4, 3, 5, 8, 6]` Output: `[1, 8]` Explanation: minimum and maximum elements of array are 1 and 8.

Problem Solved Successfully ✓

Test Cases Passed: **1111 / 1111**

Attempts: Correct / Total: **1 / 2**

Accuracy: 50%

Points Scored: **1 / 1**

Your Total Score: 101 ↑

Solve Next

Type of array, Largest in Array, First and Second Smallests

Stay Ahead With:

Build 21 Projects in 21 Days
 Build real-world ML, Deep Learning & Gen AI projects

```

1 import java.util.*;
2
3 class Solution {
4     public ArrayList<Integer> getMinMax(int[] arr) {
5         int min = arr[0];
6         int max = arr[0];
7
8         for (int i = 1; i < arr.length; i++) {
9             if (arr[i] < min) {
10                min = arr[i];
11            }
12            if (arr[i] > max) {
13                max = arr[i];
14            }
15        }
16
17        ArrayList<Integer> result = new ArrayList<>();
18        result.add(min);
19        result.add(max);
20
21        return result;
22    }
23 }
24 }

```

9) Given an array `arr[]`. Your task is to find the minimum and maximum elements in the array.
 Examples: Input: `arr[] = [1, 4, 3, 5, 8, 6]` Output: `[1, 8]` Explanation: minimum and maximum elements of array are 1 and 8.



Courses

Tutorials

Practice

Jobs

Problem

Editorial

Submissions

Comments

Output Window

Compilation Results

Custom Input

Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1115 / 1115

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Points Scored

1 / 1

Your Total Score: 102

Time Taken

0.49

Solve Next

Last index of One

Pairs with Positive Negative values

Repeated IDs

Stay Ahead With:



Build 21 Projects in 21 Days

Build real-world ML, Deep Learning & Gen AI projects

Java (21)

Start Timer

```
1 class Solution {
2     public static int largest(int[] arr) {
3         int max = arr[0];
4
5         // Traverse array and find maximum
6         for (int i = 1; i < arr.length; i++) {
7             if (arr[i] > max) {
8                 max = arr[i];
9             }
10        }
11        return max;
12    }
13 }
14
15 }
```

Custom Input

Compile & Run

Submit